PROJECT TITLE:

Road Lane Detection using Digital Image Processing

COURSE: Digital Image Processing Lab

PRESENTED BY:
Muhammad Wajid Jamal (NUM-BSEE-2022-24)
Muhammad Usman Shah (NUM-BSEE-2022-28)

PRESENTED TO:
Ms.Zulaikha Kiran & Ms.Aliza

PRESENTATION OUTLINE

- Problem Description
- Methodology & Approach
- Algorithm Block Diagram
- Step-by-Step Implementation
- Graphical User Interface (GUI)
- Results
- Findings & Analysis
- Conclusion

Problem Description

- **The Challenge:** Road lane detection is a critical component of intelligent transportation systems.
- **Objective:** To detect road lane markings (white and yellow) from camera images to assist vehicles in understanding road boundaries.
- **Focus:** The system must identify lanes while ignoring unnecessary background elements like trees, buildings, and other vehicles.
- **Input Data:** Front-view road images captured during daylight to ensure lane colors are visible.

Methodology & Approach

Tools Used:

- ✓ Python
- ✓ OpenCV

Core Technique:

- ✓ Color-based Segmentation in the HSV Color Space.

Why Use HSV?

- ➔ **Hue (H):** Helps to distinguish road surfaces from non-road areas, like grass.
- ➔ **Saturation (S):** Useful for identifying white lines on the road, which usually have low saturation levels (less than 0.3).
- ➔ **Value (V):** Filters out brightness, allowing us to isolate bright lane areas while ignoring shadows.

Methodology & Approach

White Color Conditions

S < 0.30

V > 0.70

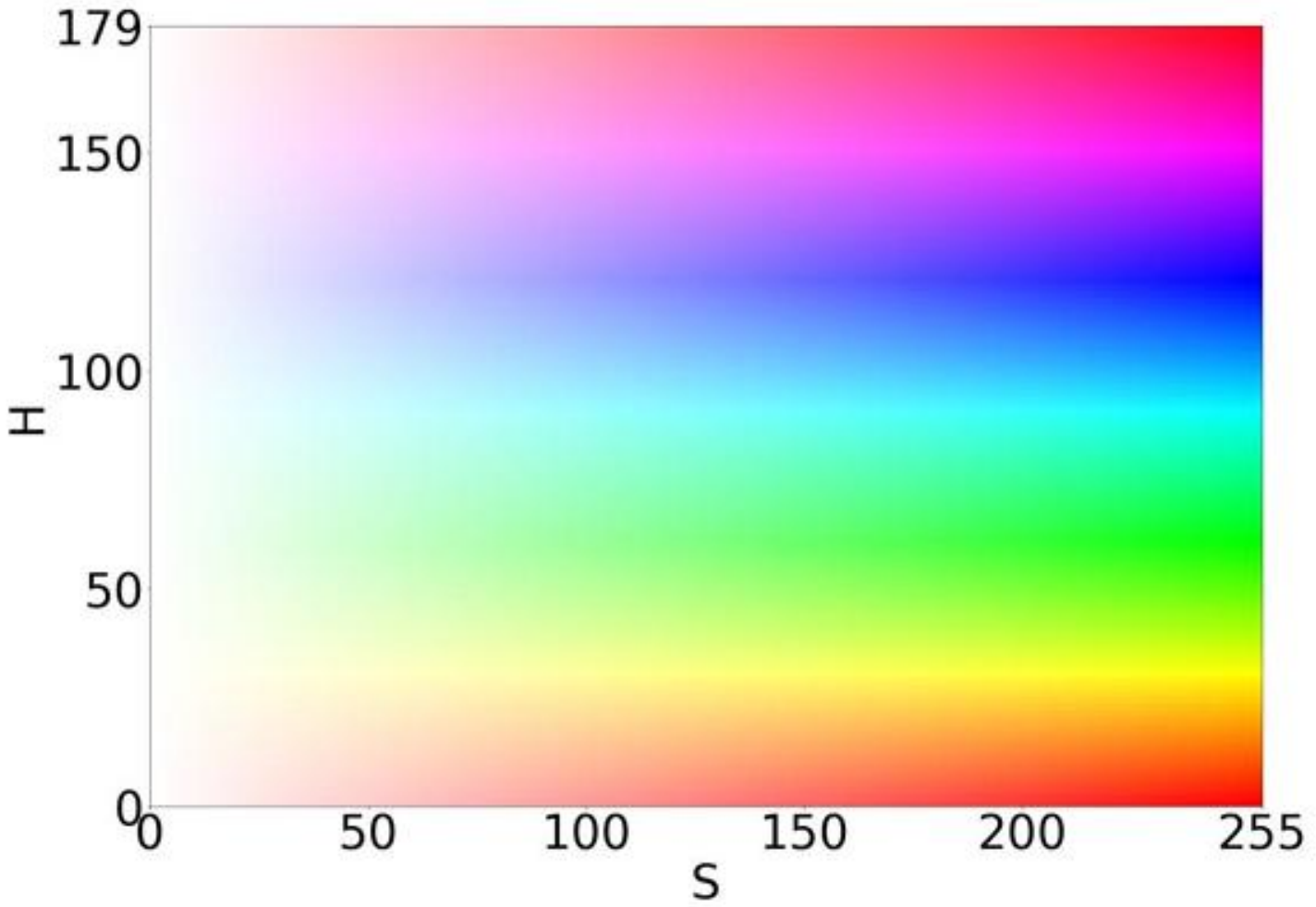
Yellow Color Conditions

H ≥ 0.024

H ≤ 0.196

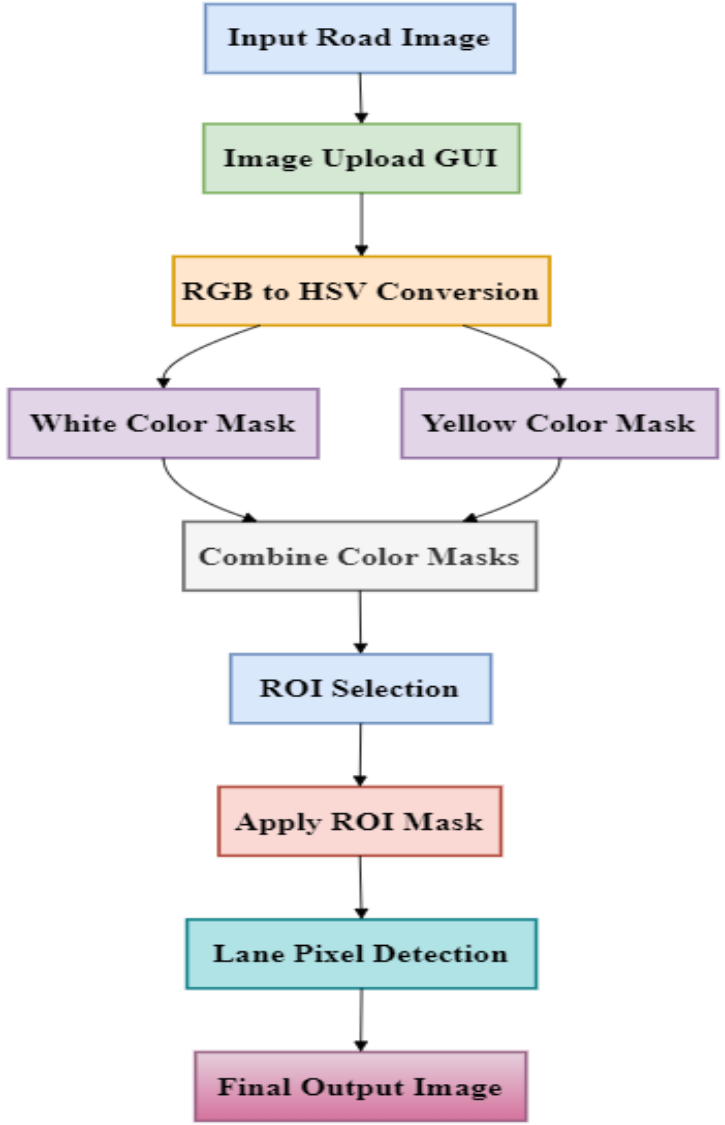
S ≥ 0.314

V ≥ 0.314



HSV color space

Algorithm Block Diagram



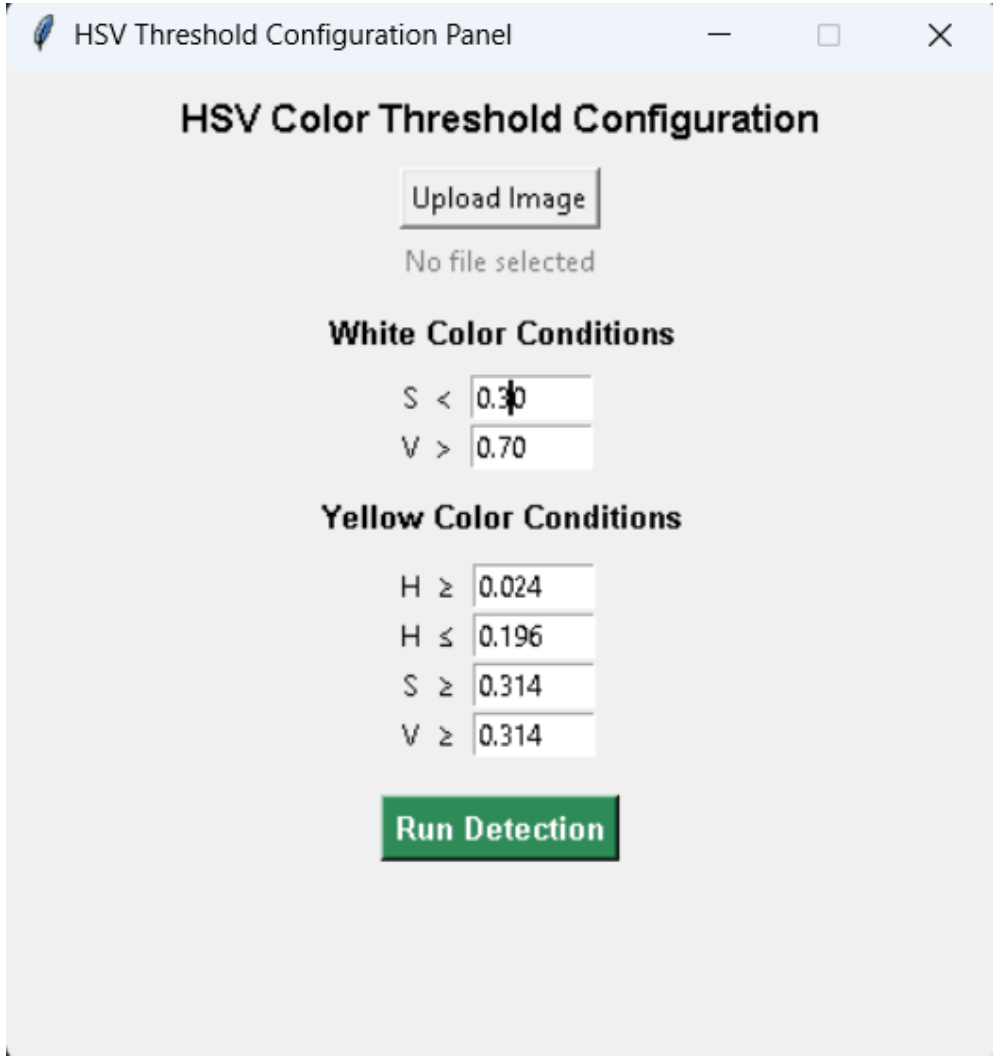
Step-by-Step Implementation

- ➔ **Image Upload:** User selects an image via the GUI.
- ➔ **Conversion:** Image is converted from RGB to HSV.
- ➔ **Mask Creation:**
  - ➔ **White Mask:** Filters for high brightness/low saturation.
  - ➔ **Yellow Mask:** Filters for specific yellow hue ranges.
- ➔ **Combination:** Both masks are combined to catch all lane pixels.
- ➔ **ROI Selection:** A manual polygon is drawn to isolate the road area.
- ➔ **Overlay:** Detected pixels are colored red on the original image.

Graphical User Interface (GUI)

- **Framework:** Built using Python's Tkinter library.
- **Features:**
  - **User-Friendly:** No programming knowledge required to operate.
  - **Automatic Processing:** Once an image is uploaded, the algorithm runs automatically.
  - **Visualization:** Displays the Original Image, Masks, ROI, and Final Result

Graphical User Interface (GUI)



Results – Input & Color Masks

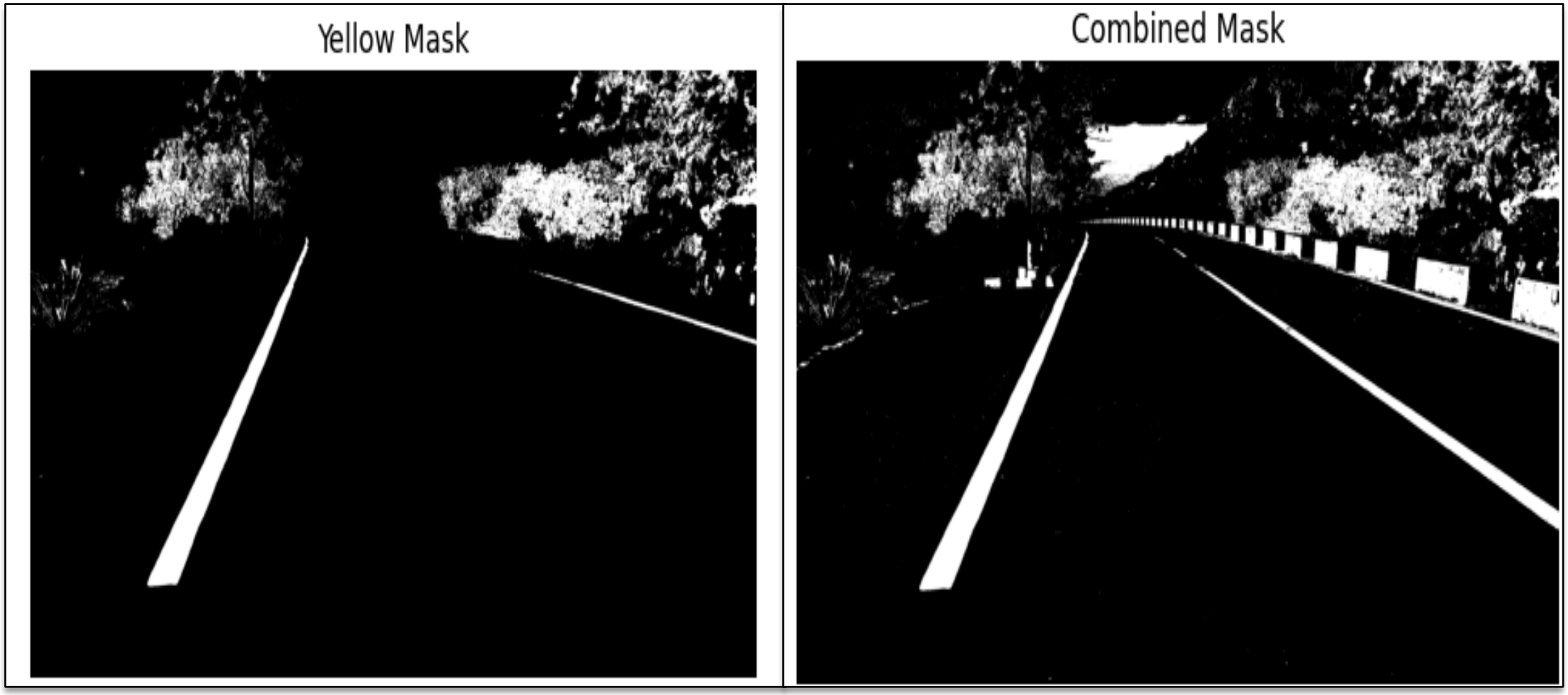




Original Image

Successfully captures white dashed lines

Results – Input & Color Masks



Identifies solid yellow lines but it also pick trees

Merges both White and Yellow Mask

ROI Mask



Final Output (Red Detection)



The Region That we have selected

Final Result



Original Image With Shadow Effect

Final Result

Findings & Analysis

Successes:

➔ The system performs efficiently on clear, daylight road scenes.

➔ Successfully detects both solid (yellow) and dashed (white) markings.

Challenges:

➔ Lighting: Performance declines in low-light or night conditions.

➔ Shadows: Bright areas within shadows can cause false detections.

Limitations:

➔ The current implementation requires manual ROI selection for each image.

Conclusion

- ➔ This project demonstrated that simple digital image processing techniques (Color Segmentation + ROI) are effective for lane detection in daylight.
- ➔ A functional GUI was created to make the tool accessible.
- ➔ It serves as a foundational step toward more complex driver assistance systems.

THANK YOU

Questions & Answers

